# PBA: 8. More on Regression

Fall 2026

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

# Where are we? Where are we going?

- **Where we have been**
  - Part I, Working with Data in R
  - Part II, Causality, Prediction, and Regression
    - Week 5: causality. Causal effects, experiments, and observational studies
    - Week 6: relationships and tidying. Correlation, our own functions, joins
    - Week 7: prediction and iteration. Election polls, for loops, prediction errors and RMSE
    - Week 9: regression and model fit. Least squares, $R^2$, test error, bias and variance

# Where are we? Where are we going?

- **Where we have been**
  - Part I, Working with Data in R
  - Part II, Causality, Prediction, and Regression
    - Week 5: causality. Causal effects, experiments, and observational studies
    - Week 6: relationships and tidying. Correlation, our own functions, joins
    - Week 7: prediction and iteration. Election polls, for loops, prediction errors and RMSE
    - Week 9: regression and model fit. Least squares, $R^2$, test error, bias and variance
- **Today: more on regression, the last week of Part II**
  - Multiple regression; ridge, lasso, and elastic net; categorical predictors, an experiment, and interactions
  - After the lecture: E5 at your own pace with the TAs

# Where are we? Where are we going?

- **Where we have been**
  - Part I, Working with Data in R
  - Part II, Causality, Prediction, and Regression
    - Week 5: causality. Causal effects, experiments, and observational studies
    - Week 6: relationships and tidying. Correlation, our own functions, joins
    - Week 7: prediction and iteration. Election polls, for loops, prediction errors and RMSE
    - Week 9: regression and model fit. Least squares, $R^2$, test error, bias and variance
- **Today: more on regression, the last week of Part II**
  - Multiple regression; ridge, lasso, and elastic net; categorical predictors, an experiment, and interactions
  - After the lecture: E5 at your own pace with the TAs
- **Where we are going**
  - Part III, Statistical Uncertainty. Next class: sampling and sampling distributions
  - Part IV, Showcase

# Roadmap

1. Multiple Regression
2. Many Predictors: Ridge, Lasso, and Elastic Net
3. Categorical Predictors and an Experiment
4. Interactions
5. In-Class Exercise E5

# 1/ Multiple Regression

# Beauty and Student Evaluations *Redux*

- Why do some instructors at universities receive high teaching evaluations scores from students while others receive lower ones?
  - Data contains instructor and course information on 463 courses taught at UT Austin (`ModernDive`; see recommended resources in syllabus).

| Variable | Description |
|---|---|
| `ID` | Identifier for courses taught at UT Austin |
| `score` | Avg. teaching score (1-5) |
| `bty_avg` | Instructor's avg. "beauty" score computed from a separate panel of six students (1-10). |
| `age` | Instructor's age |
| `cls_students` | Total number of students in the class |

Table: UT Austin Teaching Eval. Data

# UT Austin Student Eval. Data

Import the data:

```
# Don't forget to load tidyverse and ggplot2!
> evals <- read_csv("https://bit.ly/46rSa7s"); evals

# A tibble: 463 × 14
      ID prof_ID score   age bty_avg gender ethnicity  language rank  pic_outfit pic_color cls_did_eval
   <dbl>   <dbl> <dbl> <dbl>   <dbl> <chr>  <chr>      <chr>    <chr> <chr>      <chr>             <dbl>
 1     1       1   4.7    36    5    female minority   english  tenu. not formal color              24
 2     2       1   4.1    36    5    female minority   english  tenu. not formal color              86
 3     3       1   3.9    36    5    female minority   english  tenu. not formal color              76
 4     4       1   4.8    36    5    female minority   english  tenu. not formal color              77
 5     5       2   4.6    59    3    male   not minor. english  tenu. not formal color              17
 6     6       2   4.3    59    3    male   not minor. english  tenu. not formal color              35
 7     7       2   2.8    59    3    male   not minor. english  tenu. not formal color              39
 8     8       3   4.1    51    3.33 male   not minor. english  tenu. not formal color              55
 9     9       3   3.4    51    3.33 male   not minor. english  tenu. not formal color             111
10    10       4   4.5    40    3.17 female not minor. english  tenu. not formal color              40
# ℹ 453 more rows
# ℹ 2 more variables: cls_students <dbl>, cls_level <chr>
# ℹ Use `print(n = ...)` to see more rows
```

# Multiple Predictors

What if we want to model $Y$ as a function of many variables?

$$\texttt{score}_i = \alpha + \beta_1 \texttt{bty_avg}_i + \beta_2 \texttt{age}_i + \varepsilon_i$$

Why? Regression can serve two different goals:[1]

- Predict: what will $Y$ be? Better predictions (at least in-sample).
- Explain: why does $Y$ happen? Better interpretation of the coefficients as **ceteris paribus** relationships:
  - $\beta_1$ is the relationship between `bty_avg` and `score` holding age constant.
  - **Statistical control**: like subclassification (Week 5), we compare instructors of the same age. It is a first step toward causal inference.

Does this mean we should use all variables at our disposal?

- Share your thoughts on *eeclass*! (voluntary)

[1] Shmueli (2010), "To Explain or to Predict?" *Statistical Science* 25(3): 289–310.

# Multiple Regression in *R*

```
> eval_cnt <- evals |> # Center the variables for better interpretation of the intercept
    mutate(bty_cnt = bty_avg - mean(bty_avg),
           age_cnt = age - mean(age))
> mult.fit.cnt <- lm(score ~ bty_cnt + age_cnt, data = eval_cnt); mult.fit.cnt

Call:
lm(formula = score ~ bty_cnt + age_cnt, data = eval_cnt)

Coefficients:
(Intercept)      bty_cnt      age_cnt
   4.174730     0.060656    -0.003059
```

- $\widehat{\alpha} \approx 4.17$: average score when both the beauty score and age are at their respective mean values (i.e., 4.42 & 48.4).
- $\widehat{\beta_1} \approx .061$: average increase in score for additional beauty score an instructor received, **holding age fixed**.
  - Ceteris paribus association b/w aesthetic evaluation and score, independent of age.
- $\widehat{\beta_2} \approx -.003$: average decrease in score for one-year increase in age, **holding beauty score fixed**.

Q: Centering the variable won't change the beta coefficients, but z-score transformation would. Why?

# Least Squares with Multiple Regression

- How do we estimate the coefficients?
- The same exact way as before: minimize prediction error!
- Residuals (aka prediction error) with multiple predictors:

$$Y_i - \widehat{Y}_i = \text{score}_i - \widehat{\alpha} - \widehat{\beta}_1 \text{bty_avg}_i - \widehat{\beta}_2 \text{age}_i$$

- Find the coefficients that minimize the sum of the squared residuals:

$$\text{SSR} = \sum_{i=1}^{n} (Y_i - \widehat{Y}_i)^2 = \sum_{i=1}^{n} (Y_i - \widehat{\alpha} - \widehat{\beta}_1 X_{i1} - \widehat{\beta}_2 X_{i2})^2$$

# Model Fit with Multiple Predictors

- $R^2$ mechanically increases when you add variables to the regression.
  - Why? Set the new coefficient to 0, and the larger model is the old model. So its smallest SSR cannot be higher, and TSS does not change.
  - So by $R^2$, the model with the most variables always looks best.
  - But this could be overfitting! That is, our model may predict poorly on out-of-sample.
- Solution: penalize regression models with more variables.
  - Occam's razor: **simpler models are preferred**.
- Adjusted $R^2$: $R^2$ with a penalty for each predictor.

$$\text{Adjusted } R^2 = 1 - \underbrace{\frac{SSR}{TSS}}_{1-R^2} \times \frac{n-1}{n-k-1}$$

  - Week 9: $R^2 = \frac{TSS-SSR}{TSS} = 1 - \frac{SSR}{TSS}$. Here $n$: observations; $k$: predictors.
  - The factor $\frac{n-1}{n-k-1}$ grows with each predictor, so a new predictor must lower SSR enough.
  - If the added predictors don't help predict, adjusted $R^2$ goes down!

# Comparing Model Fits

- Add one predictor at a time. Row 4 adds the ages in a random order.

```
> library(broom)
> fit.bty <- lm(score ~ bty_avg, data = evals)   # the beauty score model from Week 9
> fit.size <- lm(score ~ bty_avg + age + cls_students, data = evals)   # add class size
> set.seed(1)   # sample(age): the same ages in a random order, unrelated to the score
> fit.rand <- lm(score ~ bty_avg + age + cls_students + sample(age), data = evals)
> bind_rows(glance(fit.bty), glance(mult.fit.cnt), glance(fit.size), glance(fit.rand)) |>
    select(df, r.squared, adj.r.squared)   # model df: the number of predictor coefficients
# A tibble: 4 × 3
     df r.squared adj.r.squared
  <dbl>     <dbl>         <dbl>
1     1    0.0350        0.0329
2     2    0.0378        0.0336
3     3    0.0379        0.0316
4     4    0.0388        0.0304
```

- $R^2$ rises every time, even with the ages in a random order (row 4).
- ⇝ Q: What if we observed the $adj.R^2$ decrease? (rows 3 and 4)
  - It does not lower SSR enough. By this measure, leave it out.
  - But the check is weak: by chance, 307 of 1,000 random orders correlate enough with the scores ($|t| > 1$) to raise adjusted $R^2$ (R script).
- Adjusted $R^2$ is computed from the data used to fit the model.
  - It is a quick check for overfitting. The direct check is the error on test data (Week 9). We use it in Section 2.

# Predicted Values from *R*

- Can we predict the evaluation score of a course currently being taught this semester (2026) with the following observed values?:

```
> mult.fit <- lm(score ~ bty_avg + age, data = evals)  # same model, not centered
> evals_26 <- evals |>   ## Suppose we have course 464 being taught in 2026
    select(ID, score, age, bty_avg) |>   # keep the four columns we use
    bind_rows(
      data.frame(ID = 464, score = NA, age = 36, bty_avg = 4.7)
    )
```

- The `modelr` package gives some functions that allow us to make predictions in a tidy way.
  - We could plug in values into the equation, but *R* can do this for us.
  - Let's use `add_predictions()` to predict the 2026 results:

```
> library(modelr)
> evals_26 |>
    filter(ID == 464) |>
    add_predictions(mult.fit)

# A tibble: 1 × 5
     ID score   age bty_avg  pred
  <dbl> <dbl> <dbl>   <dbl> <dbl>
1   464    NA    36     4.7  4.23
```

# Predictions from Several Models

- The `gather_predictions()` will return one row for each model passed to it with the prediction for that model:

```
> evals_26 |>
    filter(ID == 464) |>
    gather_predictions(fit.bty, mult.fit)

# A tibble: 2 × 6
  model       ID score   age bty_avg  pred
  <chr>    <dbl> <dbl> <dbl>   <dbl> <dbl>
1 fit.bty    464    NA    36     4.7  4.19
2 mult.fit   464    NA    36     4.7  4.23
```

# Predictions from New Data

- How do we predict several new courses at once?
  - We do not need to add the courses to the data. We only need a tibble with the values of the predictors.

```
> tibble(bty_avg = c(2.5, 4.5), age = 36) |>
    gather_predictions(fit.bty, mult.fit)

# A tibble: 4 × 4
  model    bty_avg   age  pred
  <chr>      <dbl> <dbl> <dbl>
1 fit.bty      2.5    36  4.05
2 fit.bty      4.5    36  4.18
3 mult.fit     2.5    36  4.10
4 mult.fit     4.5    36  4.22
```

- We use one or two predictors here so that the output is easy to read.
  - A real prediction model uses many predictors. On slide 24, we use `add_predictions()` with 27 predictors.
- But more predictors can make the predictions worse.
  - Even the random ages (slide 10) get a coefficient: 0.0017, not 0.
  - Fit with half of the courses: when the random ages raise adjusted $R^2$, the other half is predicted worse 3 times in 4 (R script).
- Section 2: with 27 predictors and 40 houses, the damage is far larger.

# **2/** Many Predictors: Ridge, Lasso, and Elastic Net

# Predicting House Prices in Ames, Iowa

- Goal: predict the sale price of a house before it sells.
  - Data: 2,904 home sales in Ames, Iowa, from 2006 to 2010.[2]

| Variable | Description |
|---|---|
| `price` | Sale price in $1,000s (median: 160) |
| `Gr Liv Area` | Living area above ground (square feet) |
| `Overall Qual` | Overall quality of materials and finish (1–10) |
| `Year Built` | Year the house was built |
| `Garage Cars` | Size of the garage (number of cars) |
| … | 23 more numeric features (lot, baths, rooms, …) |

Table: Ames Housing Data

- ⇝ Q: Which features would you use to predict the price of a house?

[2] D. De Cock (2011), *Journal of Statistics Education* 19(3); Ames, Iowa Assessor's Office. Of the 2,930 sales in the file, 2,904 have all 27 numeric features.

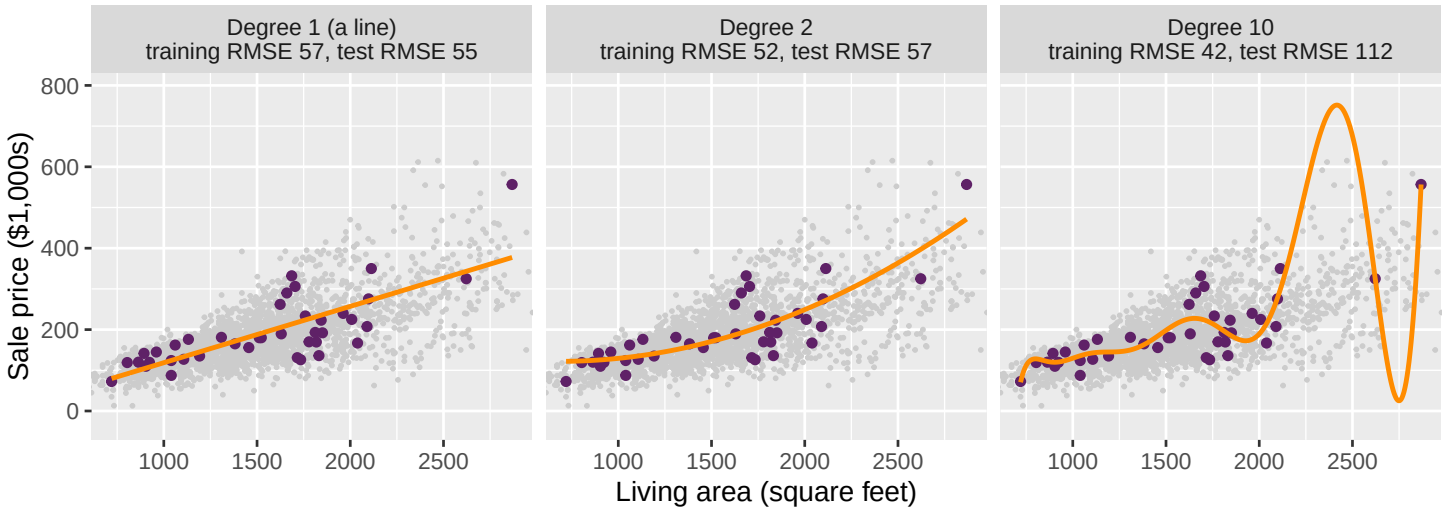
# Ames Data in *R*: 40 Training Houses

- Import the data, keep 27 numeric features, draw 40 training houses:

```
> predictors <- c("Lot Area", "Overall Qual", "Overall Cond", "Year Built",
    "Year Remod/Add", "Mas Vnr Area", "Total Bsmt SF", "Gr Liv Area", "Bsmt Full Bath",
    "Bsmt Half Bath", "Full Bath", "Half Bath", "Bedroom AbvGr", "Kitchen AbvGr",
    "TotRms AbvGrd", "Fireplaces", "Garage Cars", "Garage Area", "Wood Deck SF",
    "Open Porch SF", "Enclosed Porch", "3Ssn Porch", "Screen Porch", "Pool Area",
    "Misc Val", "Mo Sold", "Yr Sold")   # the 27 numeric features
> ames <- read_tsv("https://jse.amstat.org/v19n3/decock/AmesHousing.txt") |>
    select(SalePrice, all_of(predictors)) |>    # all_of(): the columns named in predictors
    drop_na() |>                                # 2,904 sales with complete records
    mutate(price = SalePrice / 1000)            # sale price in $1,000s
> ames |> select(price, `Gr Liv Area`, `Overall Qual`, `Year Built`, `Garage Cars`) |>
    head(3)
# A tibble: 3 × 5
  price `Gr Liv Area` `Overall Qual` `Year Built` `Garage Cars`
  <dbl>         <dbl>          <dbl>        <dbl>         <dbl>
1   215          1656              6         1960             2
2   105           896              5         1961             1
3   172          1329              6         1958             1
> set.seed(87746)
> train <- sample(nrow(ames), 40)                          # 40 training houses
# data[rows, columns], as in Week 1 (R basics); -train means all rows but train
> ames_train <- ames[train, ]; ames_test <- ames[-train, ]  # test: the other 2,864 houses
```

- The next slides fit models to these 40 houses (few on purpose, slide 17) and test them on the other 2,864.

# Polynomial Regression Can Overfit




- Purple: the 40 training houses (slide 15). Grey: the test houses.[3]
- Degree 2: $\texttt{price}_i = \alpha + \beta_1 \texttt{area}_i + \beta_2 \texttt{area}_i^2 + \varepsilon_i$.
  - Linear regression adds up $\beta\times$ (a column). $\texttt{area}_i^2$ is one more column, so this polynomial is still linear regression.

```
> lm(price ~ poly(`Gr Liv Area`, 2, raw = TRUE), data = ames_train)   # degree 2
> lm(price ~ `Gr Liv Area` + I(`Gr Liv Area`^2), data = ames_train)   # degree 2, with I()
> lm(price ~ poly(`Gr Liv Area`, 10), data = ames_train)              # degree 10
```

- Degree 10 chases single houses: best training fit, much worse test fit.

[3] Test RMSE over the 2,794 test houses inside the living-area range of the training houses. Polynomial regression: ISLR 2nd ed., Section 7.1.

# Few Houses: 27 Predictors Overfit

- In practice we train on most of the data (80/20). Here we use only 40 houses on purpose: are 27 predictors too many for so few houses?
- This loop makes the table (test: the other 2,864 houses each round):

```
> rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))   # root mean squared error (Week 9)
> few <- c("Gr Liv Area", "Overall Qual", "Year Built")
> output <- tibble(few_train = rep(NA, 200), few_test = NA,   # 1. output: 200 rows
                   all_train = NA, all_test = NA)
> for (r in 1:200) {                  # 2. sequence
    set.seed(r); i <- sample(nrow(ames), 40)   # 3. body: 40 random houses
    fit_few <- lm(price ~ ., data = ames[i, c("price", few)])         # 3 predictors
    fit_all <- lm(price ~ ., data = ames[i, c("price", predictors)])  # all 27 predictors
    output$few_train[r] <- rmse(ames$price[i], predict(fit_few, ames[i, ]))  # training RMSE
    output$few_test[r] <- rmse(ames$price[-i], predict(fit_few, ames[-i, ])) # test RMSE
    output$all_train[r] <- rmse(ames$price[i], predict(fit_all, ames[i, ]))
    output$all_test[r] <- rmse(ames$price[-i], predict(fit_all, ames[-i, ]))
  }
> output |> summarize(across(few_train:all_test, median))   # medians over the 200 samples
# A tibble: 1 × 4
  few_train few_test all_train all_test
      <dbl>    <dbl>     <dbl>    <dbl>
1      33.2     42.0      14.4     58.2
```

- 3 predictors: training RMSE 33.2, test RMSE 42.0. All 27 predictors: training 14.4, test 58.2 (in $1,000s).
- With 27 predictors and 40 houses, least squares fits the noise.

# Too Many Columns, Even with Many Houses

- Multiply every pair of predictors, for example, living area × quality.
  - Like $\texttt{area}^2$ on slide 16, each product is one more column of numbers.
  - 27 predictors + 351 products = 378 columns.
- This time, train on many houses, as usual in practice: 2,323 (80%). Test on the other 581.
  - Repeat with 200 random splits. Code: today's R script.

| Least squares with | Training RMSE | Test RMSE |
|---|---|---|
| 27 predictors | 33.3 | 33.3 |
| 378 columns | 19.1 | 71.3 |

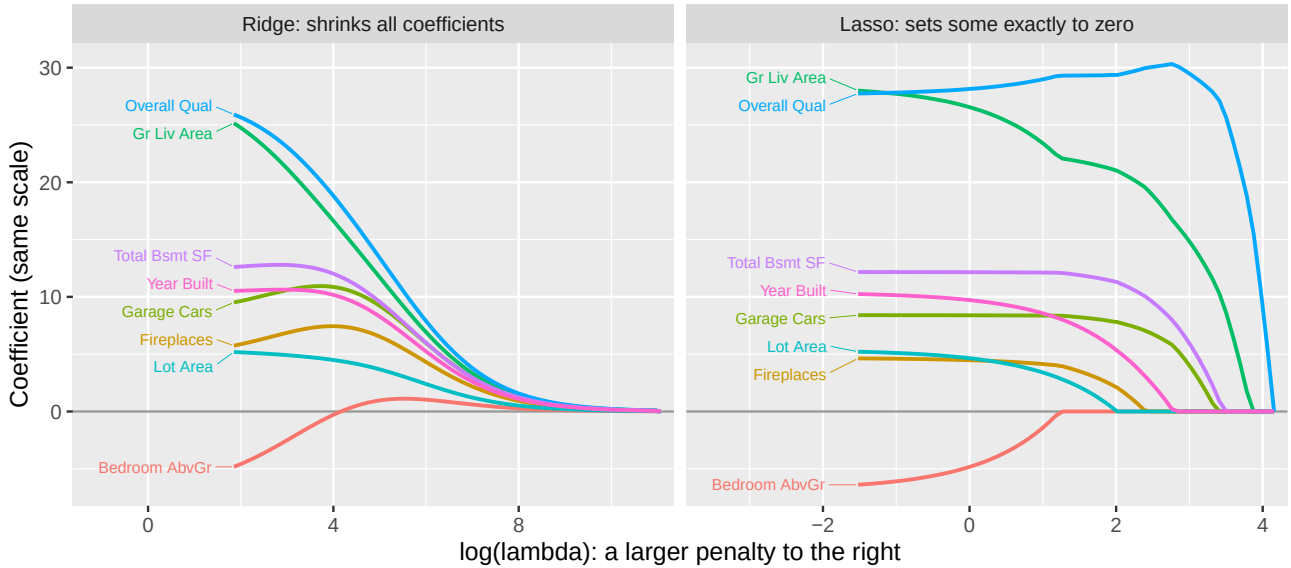
*Notes.* Price in $1,000s; medians over the 200 splits.

- Even with 2,323 training houses, the 378 columns fit the noise.
  - The training error falls, and the test error rises (in 172 of 200 splits).
  - In the worst split, the test RMSE was 12,133.
- With least squares, overfitting depends on how many coefficients we fit for the number of houses.
  - Too few houses (slide 17) or too many columns (here).

# Penalized Regression: Penalty on Large Coefficients

- Least squares picks the coefficients with the smallest SSR.
  - With few houses per coefficient, some get very large to fit the noise.
  - Garage cars: 4.0 with 2,904 houses, $-39$ to 41 with 40 (R script).
  - Here a very large coefficient may be due to chance, not a strong relation.
- Penalized regression adds a penalty on large coefficients:
  - **Ridge**: SSR + $\lambda \times$ (sum of squared coefficients)
  - **Lasso**: SSR + $\lambda \times$ (sum of the absolute values of the coefficients)
  - **Elastic net**: a mix of the two penalties.
- $\lambda$ (lambda) sets the size of the penalty: $\lambda = 0$ gives least squares.
  - A very large $\lambda$ shrinks every coefficient except the intercept toward 0, so the model predicts $\overline{Y}$, the average training price.
  - A larger $\lambda$ gives a less flexible model: a little more bias, much less variance (Week 9).
- The penalty is on the size of the coefficients.
  - The predictors are first standardized (put on the same scale), so the penalty does not depend on units (`glmnet` does this by default).

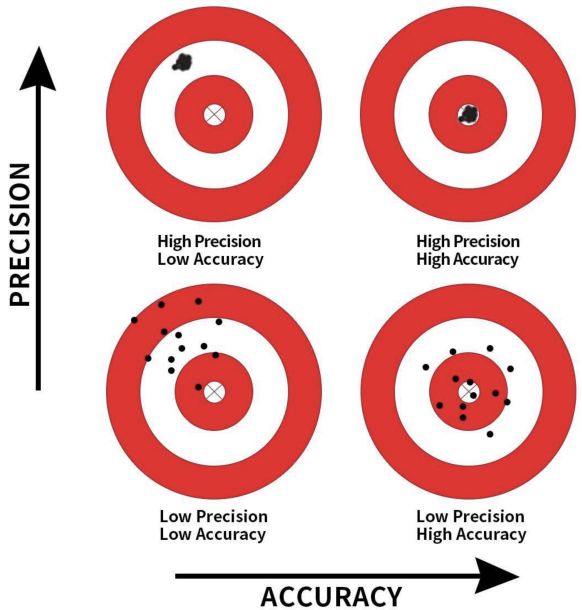
# Ridge Shrinks; Lasso Sets Some to Zero




- Moving right, $\lambda$ grows.
  - Ridge shrinks every coefficient but keeps them all.
  - Lasso sets some exactly to 0, so it also chooses predictors.
- Why exactly 0?[4]
  - Lasso: a coefficient of 0.1 adds $0.1 \times \lambda$. If the fit improves by less, it stays at 0.
  - Ridge: a coefficient of 0.1 adds only $0.01 \times \lambda$, so a small improvement keeps it.

[4] James, Witten, Hastie, and Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Sections 6.2.1–6.2.2.

# A Little Bias for Much Less Variance




Source: forecast.app

Next week we use the same target again. Each dot will then be an estimate from a different sample.

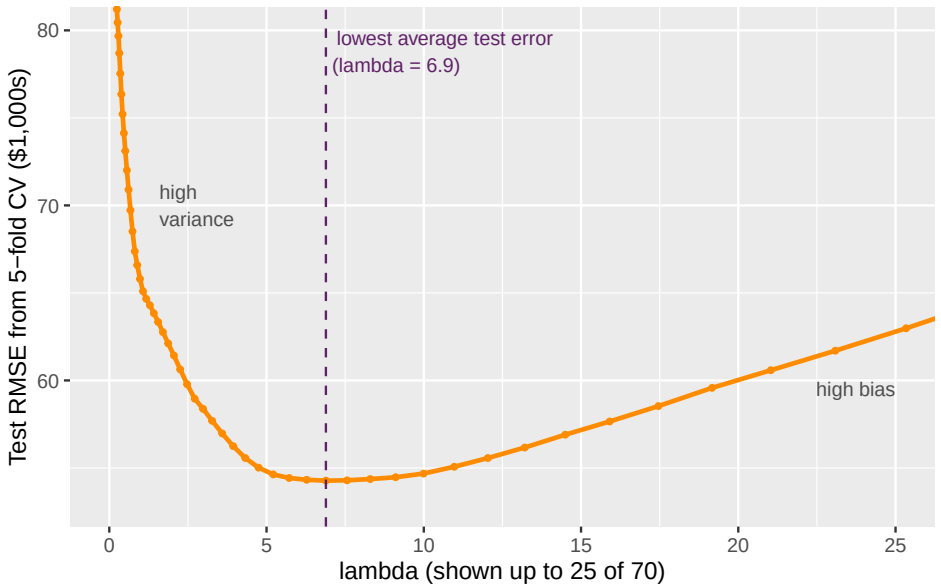
- Each dot: the predicted price of the same house from a model fit to a different set of 40 training houses (slide 17). The center is the true value.
- Least squares with 27 predictors: the dots are far apart (high variance).
  - Few houses per coefficient: noisy estimates.
  - With 3 hand-picked predictors (slide 17), the dots stay close: a simple model has low variance.
- Ridge and lasso pull the dots together: much less variance.
  - They can add a little bias, because every coefficient is pulled toward 0.[5]

| Model | Spread ($1,000s) |
|---|---|
| Least squares, 3 predictors | 14 |
| Least squares, 27 predictors | 44 |
| Ridge, 27 predictors | 21 |
| Lasso, 27 predictors | 23 |
| Elastic net, 27 predictors | 23 |

*Notes.* SD over 200 training sets, averaged over 900 test houses.

[5] ISLR 2nd ed., Section 6.2.1 and Figure 6.5. Code for the table: today's R script.

# Choosing $\lambda$: Cross-Validation




- Split the training data into 5 parts (folds).[6]
  - Fit on 4, test on the fifth, and rotate.
  - Average the 5 test errors for each $\lambda$, and pick the $\lambda$ with the lowest average.
- Each fold acts as a small test set, so we choose $\lambda$ without touching the 2,864 test houses.
  - Left: small $\lambda$, close to least squares (high variance). Right: larger $\lambda$, closer to predicting $\overline{Y}$ (high bias).
- Here (one set of 40 training houses), the chosen lasso keeps 8 of the 27 predictors.

[6] ISLR 2nd ed., Section 5.1.3 (k-fold cross-validation). In *R*: `cv.glmnet()` (next slide). Code for the figure: today's R script.

# In *R*: Penalized Models Predict Better

- Choose $\lambda$ by 5-fold cross-validation, then compute the test RMSE:

```
> library(glmnet)   # install.packages("glmnet") once
> x <- as.matrix(ames_train[, predictors])   # ames_train, ames_test, predictors: slide 15
> set.seed(87746)   # cv.glmnet() draws the 5 folds at random (as on slide 22)
> cv <- cv.glmnet(x, ames_train$price, alpha = 1, nfolds = 5, keep = TRUE)   # lasso
> cv_r <- cv.glmnet(x, ames_train$price, alpha = 0, foldid = cv$foldid)      # ridge
> cv_e <- cv.glmnet(x, ames_train$price, alpha = 0.5, foldid = cv$foldid)    # elastic net
> x_test <- as.matrix(ames_test[, predictors])
> pred <- predict(cv, newx = x_test, s = "lambda.min")   # s: lambda with lowest CV error
> rmse(ames_test$price, pred)   # test RMSE; rmse(): slide 17
[1] 41.53232
```

| 40 training houses, all 27 predictors | Median test RMSE |
|---|---|
| Least squares | 58.2 |
| Lasso | 41.6 |
| Ridge | 41.8 |
| Elastic net ($\alpha = 0.5$) | 40.9 |

*Notes.* The code above, repeated over the 200 samples of slide 17: medians of the 200 test RMSEs.

- Penalized models match our three hand-picked predictors (42.0), without our picking.
- Which one to use? None always wins: it depends on the data and the prediction task.[7]
  - Compare the models by test RMSE from cross-validation.
  - Elastic net is many models: each alpha between 0 and 1 is a different penalty.

[7] ISLR 2nd ed., Section 6.2.2. For alpha: An Introduction to glmnet.

# Three Test Houses: Which Model Predicts Closer?

- Use the trained models to predict the prices of the test houses (not used in training).

```
# Least squares, without a penalty, for comparison (the same 40 training houses)
> ls3 <- lm(price ~ ., data = ames_train[, c("price", few)])   # 3 predictors (slide 17)
> ls27 <- lm(price ~ ., data = ames_train[, c("price", predictors)])   # all 27 predictors

# cv, cv_r, cv_e: the lasso, ridge, and elastic net fits of slide 23. Here we only predict.
# predict() returns the predicted prices in a one-column matrix; [, 1] takes that column
> pred_net <- function(fit) predict(fit, newx = x_test, s = "lambda.min")[, 1]

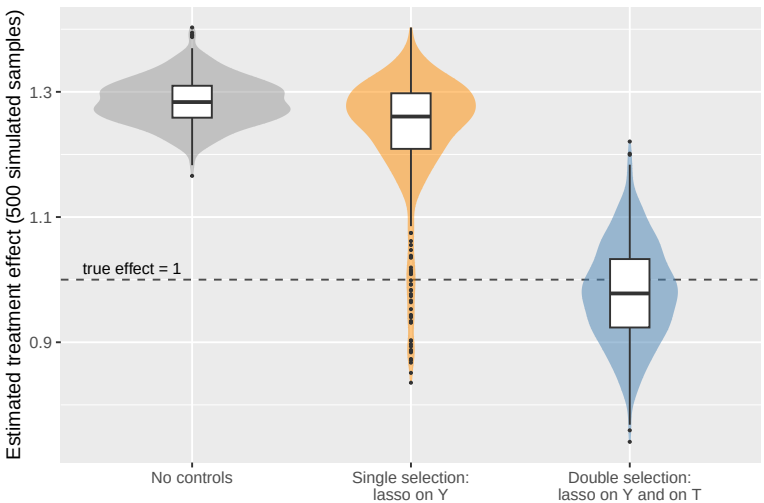
> ames_test |>   # all 2,864 test houses; var: the name of the new column
    add_predictions(ls3, var = "least_sq3") |> add_predictions(ls27, var = "least_sq27") |>
    mutate(lasso = pred_net(cv), ridge = pred_net(cv_r), elastic_net = pred_net(cv_e)) |>
    select(actual_price = price, least_sq3:elastic_net) |>
    slice(2, 52, 439)   # a cheap, a mid-priced, and an expensive house
# A tibble: 3 × 6
  actual_price least_sq3 least_sq27 lasso ridge elastic_net
         <dbl>     <dbl>      <dbl> <dbl> <dbl>       <dbl>
1          105      105.       153.  125.  127.        135.
2          160      200.       199.  204.  191.        202.
3          280      277.       294.  288.  276.        288.
```

- House 1 sold for 105 (prices in $1,000s).
  - Least squares with 27 predictors predicts 153 (high variance).
  - The penalized models predict 125 to 135: closer, but still too high.
- No model is right for every house.
  - Test RMSE (slide 23) summarizes the errors over all 2,864 test houses.

# Careful: Lasso Picks Predictors, Not Causes

- Lasso keeps the variables that help predict the outcome, whatever the reason they help.
  - So it cannot tell a **confounder**, which we must control for, from a variable that the treatment or the outcome itself changes, which we must not control for.
- A zero does not mean “not useful” either.
  - It means “not needed once the other predictors are in”. Garage cars and garage area carry almost the same information (correlation 0.89), so lasso often kept only one (60% of the 200 training sets of slide 21).
- To learn what a variable does, we need a fair comparison, such as an experiment (later today).

# Lasso for Controls: Use Double Selection




- Simulated: 200 people, 50 candidate controls, all measured before the treatment. True effect: 1.
  - 5 confounders drive $T$ strongly and $Y$ weakly.
- **Single selection** (lasso on $Y$, $T$ kept in): estimate 1.23.
  - Given $T$, the confounders barely help predict $Y$, so lasso drops them (keeps about 1 of 5).
- **Double selection**: also run lasso of $T$ on the controls, and control for every variable chosen in either step: estimate 0.98.
  - Keeps all 5.[8]
- To choose controls with lasso, keep what predicts $Y$ **and** what predicts $T$.
  - Caveat: M-bias remains. Controlling for some pre-treatment variables adds bias.

[8] Belloni, Chernozhukov, and Hansen (2014), *Review of Economic Studies* 81(2): 608–650. Simulation: 500 samples; $\lambda$ by 5-fold CV (`lambda.min`). Code: today's R script.

# 3/ Categorical Predictors and an Experiment

# Binary Predictors

$$Y_i = \alpha + \beta X_i + \varepsilon_i$$

- When the predictor $X_i$ is **binary**:
  - Intercept $\widehat{\alpha}$ is the average outcome in the $X = 0$ group.
  - Slope $\widehat{\beta}$ is the difference-in-means of $Y$ between $X = 1$ group and $X = 0$ group.

$$\widehat{\beta} = \overline{Y}_{\text{treated}} - \overline{Y}_{\text{control}}$$

- If there are other predictors, this becomes the difference in means controlling for them.

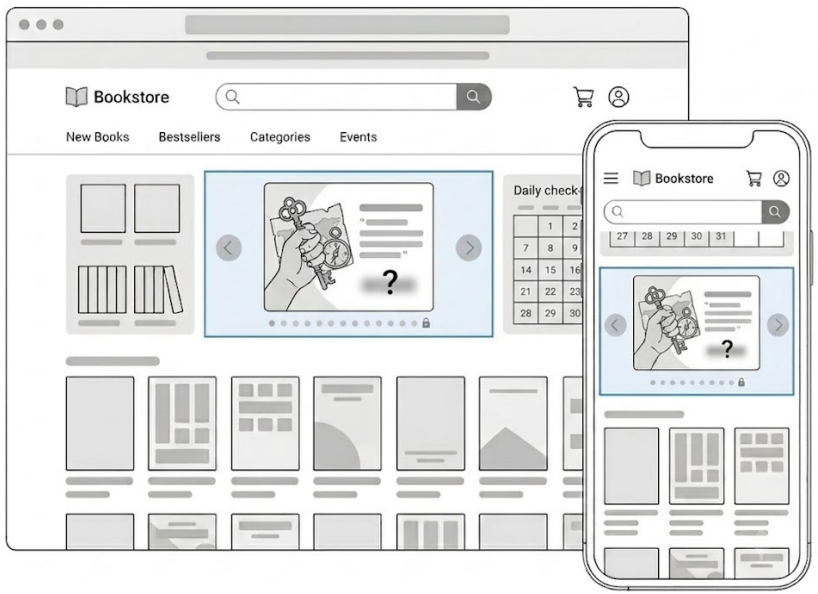
# A Research Example: Blind Lead-In (BLI) Ads




- An interactive ad at a large Korean online bookstore: swipe through 6 to 29 panels (median 12).
  - The book is shown only on the last panel: a blind lead-in (BLI) ad.
- Viewers swipe at their own pace and can stop at any panel.
  - Only those who reach the end see the book.[9]

[9] Yoon, Yoo, Kim, Kim, and Lee, "Interactive Blind Lead-In Ads and Purchase Conversion," *Journal of Interactive Marketing*, forthcoming.

# A Field Experiment: Slider Ad vs. Banner




Schematic of the bookstore's daily check-in page, where the ads ran.

- Same study: 109,156 customers, 80 days, 76 new books.
- Each customer was randomly assigned to see:
  - the **slider ad** (book shown at the end), or
  - a **static banner** (book shown at once).
- Outcome: did the customer buy the advertised book within 30 days? (1 or 0)
- Q: Does the slider ad sell more books than the banner?

# Ad Experiment Data (Synthetic)

- The real customer records are under a nondisclosure agreement (NDA) with the retailer.
  - So we use a **synthetic** version, simulated to keep the directions of the real results.[10]

| Variable Name | Description |
|---|---|
| `treat_ind` | 1 if assigned to the slider ad, 0 if assigned to the banner |
| `finished` | 1 if the customer swiped to the last panel (always 0 for the banner) |
| `purchase` | 1 if the customer bought the advertised book within 30 days |

```
# today's data are in the course data package pbadata (update it once: see the top of the R script)
> library(pbadata)
> bli <- bli_experiment   # one row per customer visit: 4,000 synthetic customers, 76 days
> bli |> head()
# A tibble: 6 × 6
  customer date         book treat_ind finished purchase
     <int> <date>      <int>     <int>    <int>    <int>
1        8 2017-06-13 100001         0        0        0
2        9 2017-06-13 100001         0        0        0
3       18 2017-06-13 100001         1        0        0
4       20 2017-06-13 100001         1        0        0
5       27 2017-06-13 100001         0        0        0
6       29 2017-06-13 100001         0        0        0
```

[10] From the study's replication package (deposited in Harvard Dataverse; public release forthcoming).

# Linear Regression for Experimental Data

- Difference in means by assignment:

```
> bli |>
    group_by(treat_ind) |>
    summarize(purchase = mean(purchase)) |>
    pivot_wider(names_from = treat_ind, values_from = purchase) |>
    mutate(ITT = `1` - `0`)
# A tibble: 1 × 3
     `0`    `1`     ITT
   <dbl>  <dbl>   <dbl>
1 0.0127 0.0143 0.00164
```

- The same number from `lm()`:

```
> lm(purchase ~ treat_ind, data = bli) |> coef() |> round(5)
(Intercept)   treat_ind
    0.01266     0.00164
```

- Same `lm()` as for prediction.
  - Under random assignment, the coefficient on `treat_ind` is the effect of being assigned the slider ad, the **intent-to-treat (ITT)** effect.
  - +0.16 percentage points (1.27% to 1.43%).

# Who Finishes the Ad?

- Among visits assigned to the slider ad, compare those that finished with those that did not:

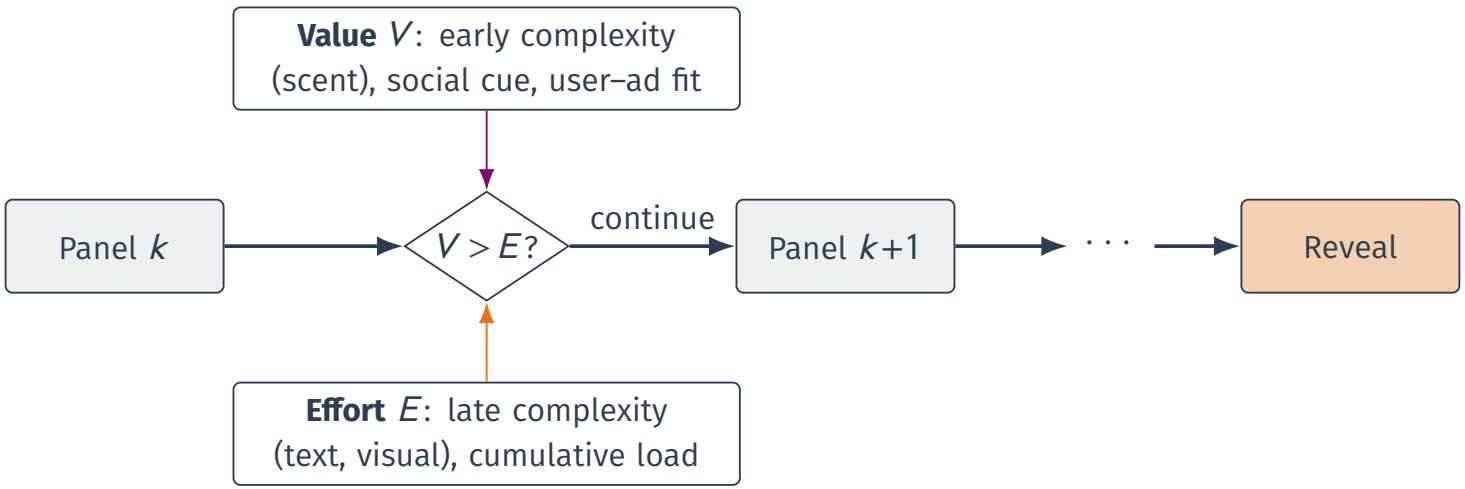
```
> bli |>
    filter(treat_ind == 1) |>
    group_by(finished) |>
    summarize(n = n(), purchase = mean(purchase))
# A tibble: 2 × 3
  finished     n purchase
     <int> <int>    <dbl>
1        0 36750  0.00955
2        1  1242  0.155
```

- Only 1,242 of 37,992 slider visits (3.3%) reached the last panel.
- Visits that finished bought far more: 15.5% vs. 1.0%.
  - A gap of 14.5 percentage points.
- Q: Is 14.5 points the effect of finishing the ad?
  - No. Viewers chose to finish, and those who finish may be likely buyers anyway: not a fair comparison (Week 5).
- The fair comparison is by assignment: +0.16 points (slide 31)

  ⇝ Finishing matters.

# How Can We Explain Finishing?

- Few viewers finish: 3.3% of all views, from 0.7% to 7.7% across the 74 ads.
- **Cost of Progress (CoP)**: a viewer continues while value $V$ > effort $E$.




- **Stage asymmetry**: the same complexity (text, color) is *value early, effort late*.
  - Early text: information scent (what the story is about).
  - Late text and color: effort.
- A human face is a social cue: value at any stage.
- Here the goal is to explain, not to predict: the variables come from theory, and the data check the signs.

# Ad Data: 74 Slider Ads

- To check the theory, we use the ad data: one row per slider ad (real data; book titles removed).[11]

| Variable Name | Description |
|---|---|
| `completion` | % of views that reached the last panel |
| `first_chars` | Number of characters of text on the first panel |
| `late_chars` | Characters of text per panel on the late panels (before the last) |
| `late_color` | How colorful the images on the late panels are |
| `first_face` | 1 if a human face appears on the first panel (also `middle_face`, `late_face`) |
| `any_face` | 1 if a human face appears on any panel (created below) |
| ... | 12 more: date, number of panels, tone and story of the text, author photo |

```
> bli_ads <- bli_ads |>   # add any_face: 1 if a human face appears on any panel, 0 if on none
    mutate(any_face = if_else(first_face + middle_face + late_face > 0, 1, 0))
> bli_ads |> select(ad, completion, first_chars, late_chars, late_color, any_face) |> head()
# A tibble: 6 × 6
  ad    completion first_chars late_chars late_color any_face
  <chr>      <dbl>       <int>      <dbl>      <dbl>    <dbl>
1 Ad_01      0.679          23       35.6      0.360        1
2 Ad_02      1.77           21       39.1      0.453        1
3 Ad_03      3.00           21       32.6      0.524        1
4 Ad_04      4.74           79       94.8      0.433        1
5 Ad_05      1.48           47       75.8      0.592        1
6 Ad_06      1.88           41       45.8      0.537        0
```

[11] From the study's replication package.

# Regression on 74 Ads: Do the Signs Match?

```
> ads <- bli_ads |>  # completion: % of views that reached the last panel
    # z-scores of the 13 continuous features (1 = one standard deviation); any_face stays 1 or 0
    mutate(across(total_slides:late_story, ~ (.x - mean(.x)) / sd(.x)))
> lm(completion ~ first_chars + late_chars + late_color + any_face, data = ads) |>
    coef() |> round(3)
(Intercept) first_chars  late_chars  late_color    any_face
      3.434       0.196      -0.155      -0.355       0.210
```

- The signs follow the theory, each holding the other features fixed (ceteris paribus, slide 6):
  - More text on the first panel: higher completion (scent).
  - More text and more colorful images on the late panels: lower completion (effort).
  - A face on any panel: higher completion (by 0.2 percentage points).
- Ad features were not randomly assigned, so these are associations, not causal effects (Week 5).

# The Survey Data

- A follow-up survey for the same study: 240 participants each viewed 10 slider ads. They could quit an ad at any panel.
- For each ad:
  - `finished`: 1 if they swiped to the last panel (recorded, not asked)
  - `experience`: asked after the ad, agreement with "I am having a good time right now," 1 (strongly disagree) to 5 (strongly agree, the best)
  - `purchase_intent`: how likely are they to buy the book? 1 to 5
  - `genre_match`: 1 if the book's genre is one they prefer
- Data: de-identified responses from the same replication package. Rows for books a participant had already read are dropped.

```
> survey <- bli_survey   # one row per participant and ad
> survey |> head()
# A tibble: 6 × 6
  person  book finished experience purchase_intent genre_match
   <int> <int>    <int>      <int>           <dbl>       <int>
1      1     2        1          3               2           0
2      1     3        1          4               3           0
3      1     6        1          3               2           0
4      1    10        1          1               1           0
5      1    11        1          2               2           0
6      1    17        0          1               1           0
```

# Categorical Variables in Regression

- We often have **categorical variables**, e.g.,:
  - Experience of viewing an ad: 1 (strongly negative), 2, 3, 4, 5 (strongly positive)
  - Book advertised: 76 titles
- How to incorporate into regression: create a **series of binary variables** (also called **dummy variables**)

| Unit | Experience | Exp. 1 | Exp. 2 | Exp. 3 | Exp. 4 | Exp. 5 |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 2 | 2 | 0 | 1 | 0 | 0 | 0 |
| 3 | 3 | 0 | 0 | 1 | 0 | 0 |
| 4 | 4 | 0 | 0 | 0 | 1 | 0 |
| 5 | 5 | 0 | 0 | 0 | 0 | 1 |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |

- Then include **all but one** (i.e., baseline) of these binary variables:

$$\text{intent}_i = \alpha + \beta_1 \text{Exp2}_i + \beta_2 \text{Exp3}_i + \beta_3 \text{Exp4}_i + \beta_4 \text{Exp5}_i + \varepsilon_i$$

# Interpreting Categorical Variables

$$\text{intent}_i = \alpha + \beta_1 \text{Exp2}_i + \beta_2 \text{Exp3}_i + \beta_3 \text{Exp4}_i + \beta_4 \text{Exp5}_i + \varepsilon_i$$

- $\widehat{\alpha}$: average purchase intention in the omitted group/baseline (experience 1, strongly negative).
- $\widehat{\beta}$ coefficients: average difference between each group and the baseline.
  - $\widehat{\beta_1}$: average difference in purchase intention between experience level 2 and level 1.
  - $\widehat{\beta_2}$: average difference in purchase intention between experience level 3 and level 1.

- Q: how do we interpret $\widehat{\beta_4}$?

# Regression with Categorical Variables

- Convert the categorical variable into a `factor`; the first level is the baseline. Without `factor()`, *R* treats 1 to 5 as numbers and fits one straight line:

```
> survey <- survey |>   # levels 1 to 5; level 1 (strongly negative) is the baseline
    mutate(experience = factor(experience))
```

- Regress `purchase_intent` on the factor variable:

```
> fit.exp <- lm(purchase_intent ~ experience, data = survey); fit.exp

Call:
lm(formula = purchase_intent ~ experience, data = survey)

Coefficients:
(Intercept)  experience2  experience3  experience4  experience5
      1.170        0.658        1.461        2.109        2.911
```

- A better experience goes with higher purchase intention: views rated 5 score 2.9 points higher than views rated 1 (on a 1–5 scale).

# What *R* Does Internally w/ Factors in `lm()`

- The factor/categorical variable, `experience`:

```
> survey |> select(experience) |> pull() |> head()
[1] 3 4 3 1 2 1
Levels: 1 2 3 4 5
```

- Notice, *R* automatically created a set of indicator variables for each level except the baseline:

```
> model.matrix(fit.exp) |> head()
  (Intercept) experience2 experience3 experience4 experience5
1           1           0           1           0           0
2           1           0           0           1           0
3           1           0           1           0           0
4           1           0           0           0           0
5           1           1           0           0           0
6           1           0           0           0           0
```

# Back to the Experiment: Book Dummies as Controls

- Add the book advertised on each day (76 books) as a categorical predictor:

```
> lm(purchase ~ treat_ind + factor(book), data = bli) |> coef() |> head(2) |> round(5)
(Intercept)   treat_ind
    0.01714     0.00165
```

- `factor(book)` adds 75 binary variables (76 books, one baseline).
- Under randomization, the control barely moves the estimate: 0.00164 to 0.00165.
    - Randomization, not the control, makes the comparison fair.

# 4/ Interactions

# Interactions: When One Effect Depends on Another

- Does the effect of one predictor depend on another?
  - Is the gap in purchase intention between finished and unfinished ads larger when viewing felt good?
- Add the product of the two predictors, an **interaction**:

$$Y_i = \alpha + \beta_1 X_i + \beta_2 Z_i + \beta_3 (X_i \times Z_i) + \varepsilon_i$$

  - The effect of $X$ is $\beta_1$ when $Z = 0$, and $\beta_1 + \beta_3$ when $Z = 1$.
  - With a categorical $Z$, there is one interaction for each level except the baseline.
  - In *R*, `x:z` is the interaction $X \times Z$.

# Finishing × Experience in *R*

- The model: `finished` and its interaction with each experience level. Person and book dummies compare views of the same person and of the same book:

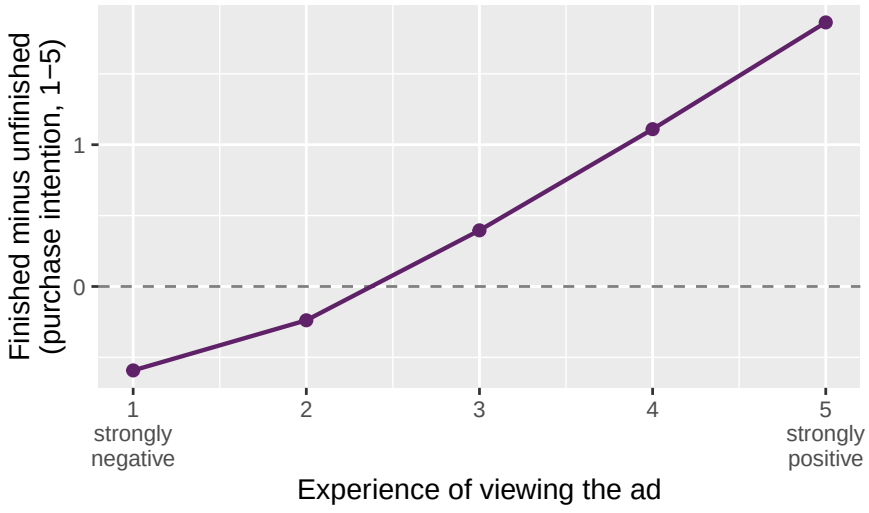
```
> fit.int <- lm(purchase_intent ~ finished + finished:experience + genre_match +
                  factor(person) + factor(book), data = survey)
> coef(fit.int)[c("finished", paste0("finished:experience", 2:5))] |> round(3)
            finished finished:experience2 finished:experience3
              -0.592                0.354                0.988
finished:experience4 finished:experience5
               1.702                2.454
```

- Gap between finished and unfinished ads at experience level $k$ = `finished` + `finished:experience`$k$:

| Experience | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| Finished − unfinished | −0.59 | −0.24 | +0.40 | +1.11 | +1.86 |

- Finishing goes with higher purchase intention only when viewing the ad felt good.

# Plotting the Interaction




- Each point is the finished-minus-unfinished gap at one experience level (slide 43):

```
> effects <- function(fit) {   # the effect of finishing at each experience level
    b <- coef(fit)
    extra <- b[paste0("finished:experience", 2:5)]   # extra effect at levels 2 to 5
    tibble(experience = 1:5, effect = b[["finished"]] + c(0, extra))
  }
> ggplot(effects(fit.int), aes(x = experience, y = effect)) + geom_line() + geom_point()
```

- Without the interaction, the model would force the same gap at every level: a flat line.

# 5/ In-Class Exercise E5

# In-Class Exercises: Background

- You are working for an investor who is thinking of building a new condominium in NYC and wants to predict the market value before making any decision.
    - The new condominium will be located in Manhattan and will be 220,000 square feet large with 300 units.

- The NYC `housing` dataset contains some information about the condominiums located in New York City and thus can be used to make the prediction. Some relevant variables:
    - `Market.Value.per.SqFt`: total market value per square foot of the land and building
    - `Boro`: the location (borough) of the condominium in NYC (e.g., Manhattan)
    - `Total.Units`: total number of units in the building
    - `Gross.SqFt`: the size of the building in square foot

# In-Class Exercises: Task

- Predict the market value per square foot of the new condominium with two regression models:
  - Model 1: the location (`boro`) only.
  - Model 2: the location, the size (`SqFt`), and the number of units (`units`).
- Compare the two models, and tell the investor whether to build.
  - To get started, we'll give you a few clues on the following slide.

# In-Class Exercises: Instructions

1. Import the data using `read_csv()` from `"https://bit.ly/49QWVuc"` and store the resulting tibble as `housing`. Using `select()` and `rename()`, keep 'Market.Value.per.SqFt', 'Boro', 'Total.Units', and 'Gross.SqFt' renamed as 'value', 'boro', 'units', and 'SqFt', respectively, for the subsequent analysis. Don't forget to overwrite `housing` with the subsetted data.
2. Specify two regression models where you consider 1) only the location (i.e., `boro`) as the predictors and 2) the location as well as the size (i.e., `SqFt`) & units (i.e., `units`) as the predictors. Write out the equations for the two models.
3. Estimate the models using `lm()` and interpret the coefficients.
4. Compare the 'fit' of the two models by looking at the adjusted $R^2$ with `glance()`. That is, do size and number of units 'add value' to our prediction?
5. Predict the market value of the condominium using `modelr` functions. What is the value of the condominium predicted to be given the specifications on slide 45?
6. The investor wants to build only if the valuation is predicted to be higher than 140 per Sq Ft. Should you recommend investing or not?

# What Comes Next?

- Today: multiple regression; ridge, lasso, and elastic net; categorical predictors; regression for an experiment; interactions. Appendix: difference-in-differences as a regression.
- Next class: **sampling and sampling distributions**. Complete the Required reading in the weekly schedule beforehand.
- Check the weekly schedule for the deadlines. Problem Set 2 (IMS Ch. 8) is released today.
- E5 (today's exercise): submit on eeclass by 23:59 today.

# Enjoy Your Weekends! :)

*Contact Information:*
jaewon.yoo@iss.nthu.edu.tw
https://j1yoo.github.io/



# Compare by Assignment, Not by Choice

- Viewers chose to finish.
  - Those who finish may be likely buyers anyway, so comparing groups that people choose into is not fair (Week 5).
- Random assignment gives the fair comparison. Few viewers finish, so the effect of assignment is diluted:

$$\text{effect of assignment} \approx \text{effect of finishing} \times \text{share who finish}$$

  - This needs one assumption: for viewers who would quit the slider ad early, the slider ad and the banner give the same chance of buying.
- So the effect of finishing $\approx 0.00164/0.0327 \approx 0.050$: about 5 percentage points.
  - A third of the 14.5-point gap.
  - It is the effect for viewers who finish, compared with the banner.

# Difference-in-Differences as a Regression

- In Week 7, we computed DiD by hand with the minimum wage data: about 2.45 workers per restaurant.
- An interaction (slide 42) gives the same number:

$$Y_{it} = \alpha + \beta_1 \text{NJ}_i + \beta_2 \text{Post}_t + \beta_3 (\text{NJ}_i \times \text{Post}_t) + \varepsilon_{it}$$

```
> minwage <- as_tibble(read.csv(url("https://bit.ly/3QZmodH")))   # same data as before
> long <- minwage |>
    mutate(id = 1:n(), nj = if_else(location == "PA", 0, 1),
           fte_before = fullBefore + 0.5 * partBefore,
           fte_after = fullAfter + 0.5 * partAfter) |>
    pivot_longer(c(fte_before, fte_after), names_to = "period", values_to = "fte") |>
    mutate(post = if_else(period == "fte_after", 1, 0))
> lm(fte ~ nj * post, data = long) |> coef() |> round(2)   # nj * post: nj + post + nj:post
(Intercept)          nj        post     nj:post
      20.30       -3.01       -2.13        2.45
```

| | |
|---|---|
| `(Intercept)` | Average FTE in PA before the law |
| `nj` | NJ minus PA before the law |
| `post` | PA's change: the change we predict for NJ without the law |
| `nj:post` | NJ's change minus PA's change: the DiD |

# Two-Way Fixed Effects

- **Fixed effects** are dummies. **Two-way fixed effects (TWFE)** use one set of dummies for the units and one set for the time periods:

$$Y_{it} = \alpha_i + \lambda_t + \beta_3(\text{NJ}_i \times \text{Post}_t) + \varepsilon_{it}$$

- $\alpha_i$: one intercept for each restaurant. $\lambda_t$: one intercept for each period. Same idea as `factor(person) + factor(book)` on slide 43.

```
> fit.twfe <- lm(fte ~ nj:post + post + factor(id), data = long)
> coef(fit.twfe)[c("post", "nj:post")] |> round(2)
   post nj:post
  -2.13    2.45
```

- `factor(id)` adds 357 binary variables (358 restaurants, one baseline).
  - We leave out `nj` because the restaurant dummies already contain it.
- DiD is the design. TWFE is one estimator.
  - It still needs parallel trends.
  - All NJ restaurants are treated at one date. With different dates, TWFE is a weighted average of many DiD comparisons, and some use already-treated units as controls (Goodman-Bacon, 2021).